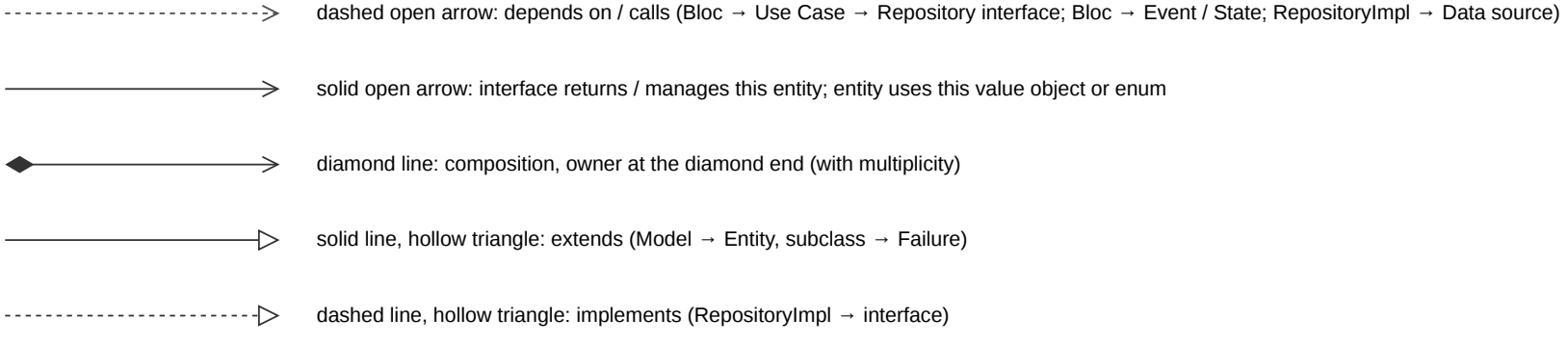
Smart Gym: Clean Architecture + BLoC (Reso Coder structure)

Every feature page reads left to right: **Event / State, Bloc** (presentation) → **Use Cases, Repository** (domain) → **RepositoryImpl, Data Sources, Model** (data) → **Entity**.

Every use case is *UseCase<Type, Params>* returning *Future<Either<Failure, Type>>*. Models extend Entities and name the Firestore collection they map to. Page C = core classes, page D = injection container (get_it).

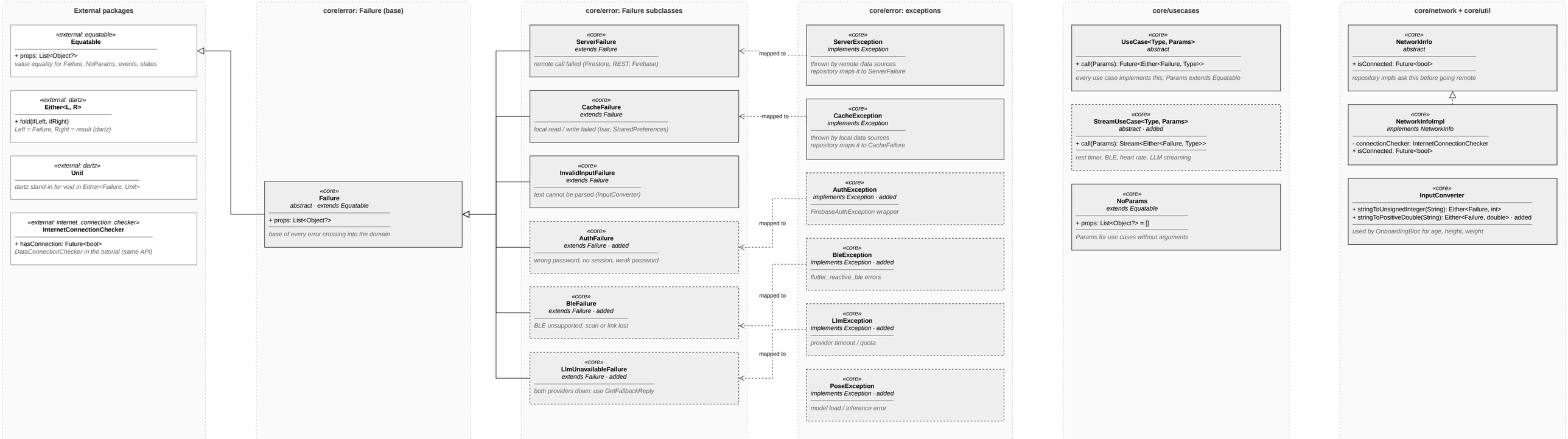
«bloc» «event» «state» «use case» «interface»

«repository impl» «data source» «model» «entity»



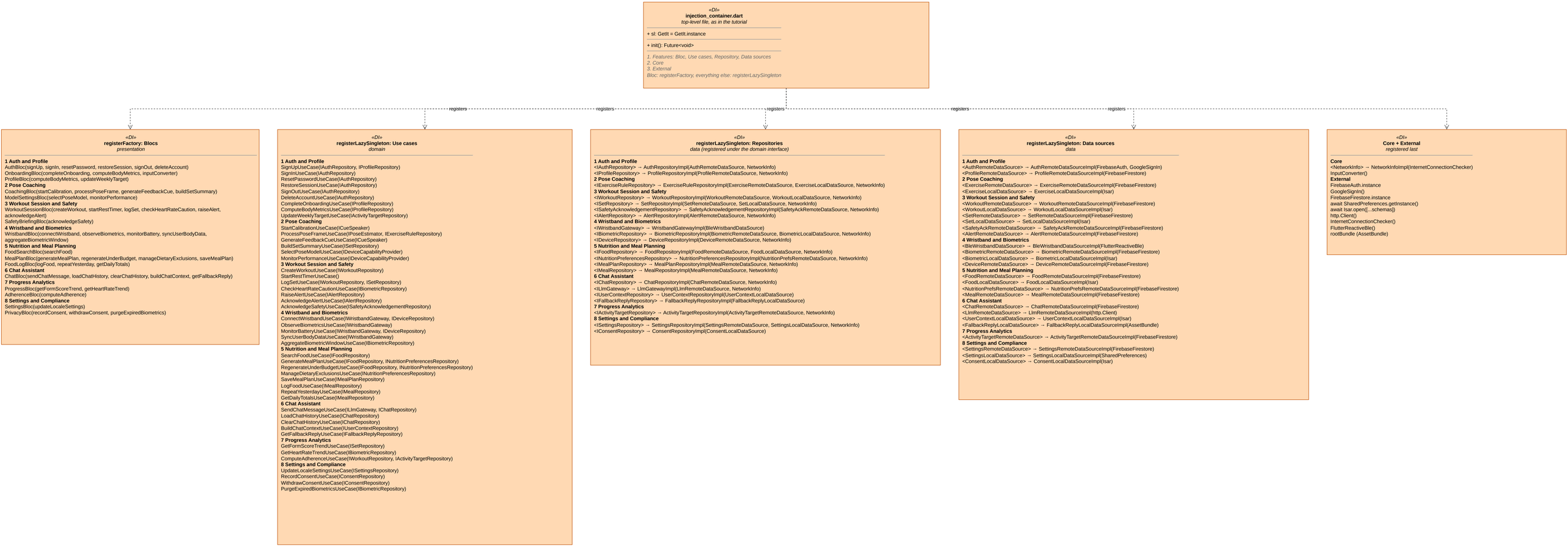
C Core: error, usecases, network, util (Reso Coder Clean Architecture)

lib/core/*: shared by every feature. Dashed border = added for Smart Gym on top of the tutorial (tutorial has Server/Cache/InvalidInput only, UseCase, NoParams, NetworkInfo, InputConverter)



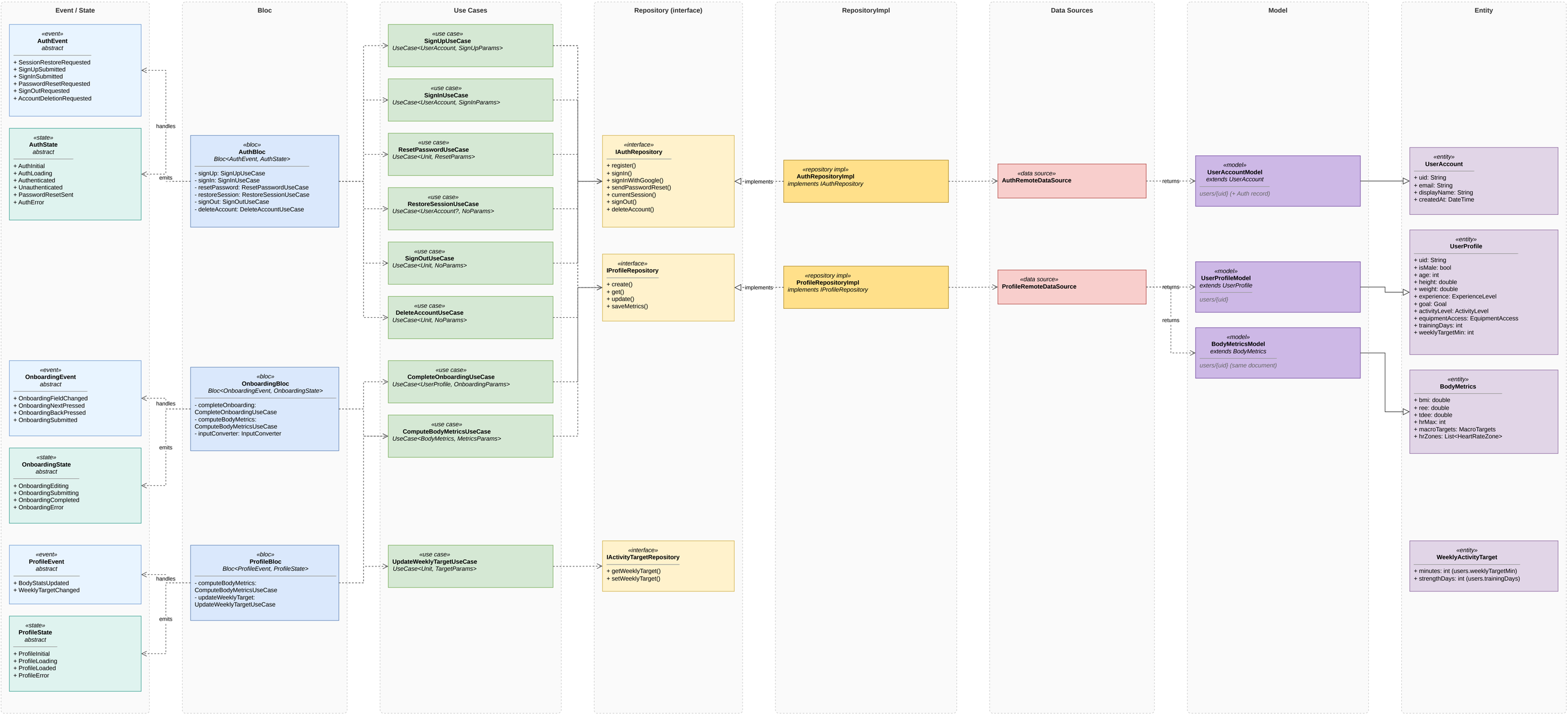
D Dependency Injection: injection_container.dart (get_it service locator)

Same shape as the Reso Coder tutorial: one global sl = GetIt.instance, one init(), grouped by feature; Blocs are factories, everything else is a lazy singleton



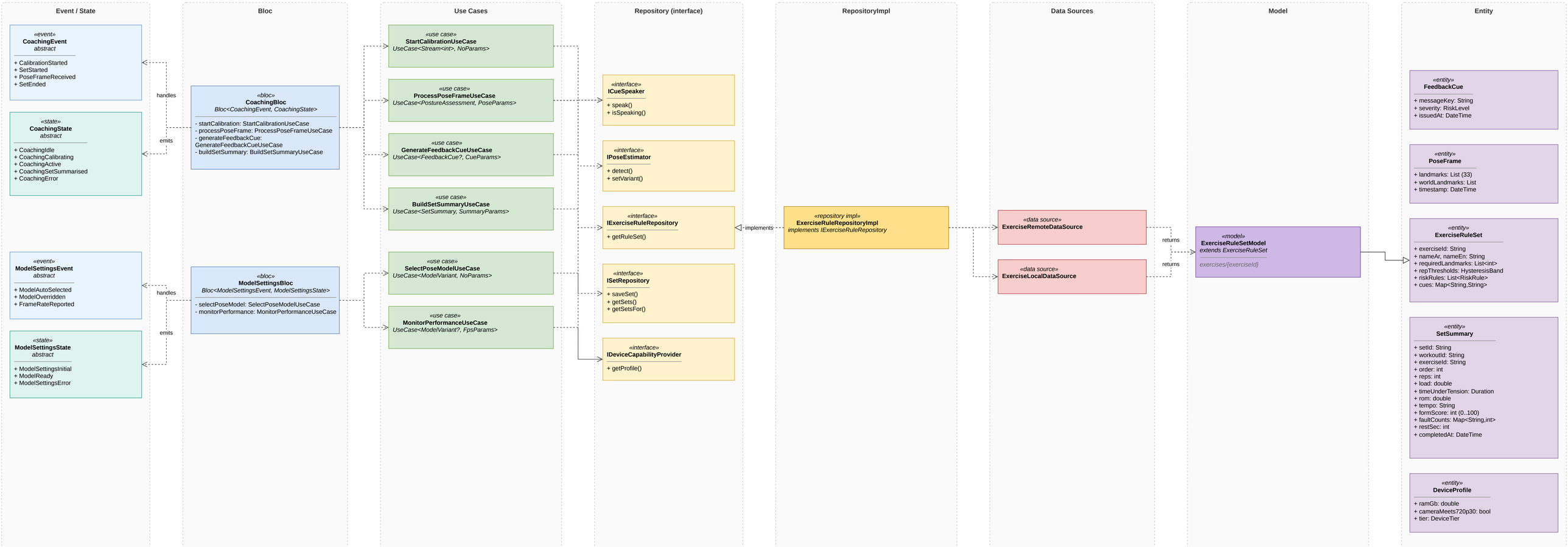
1 Auth and Profile

Accounts, sign-in, onboarding, body metrics, weekly target



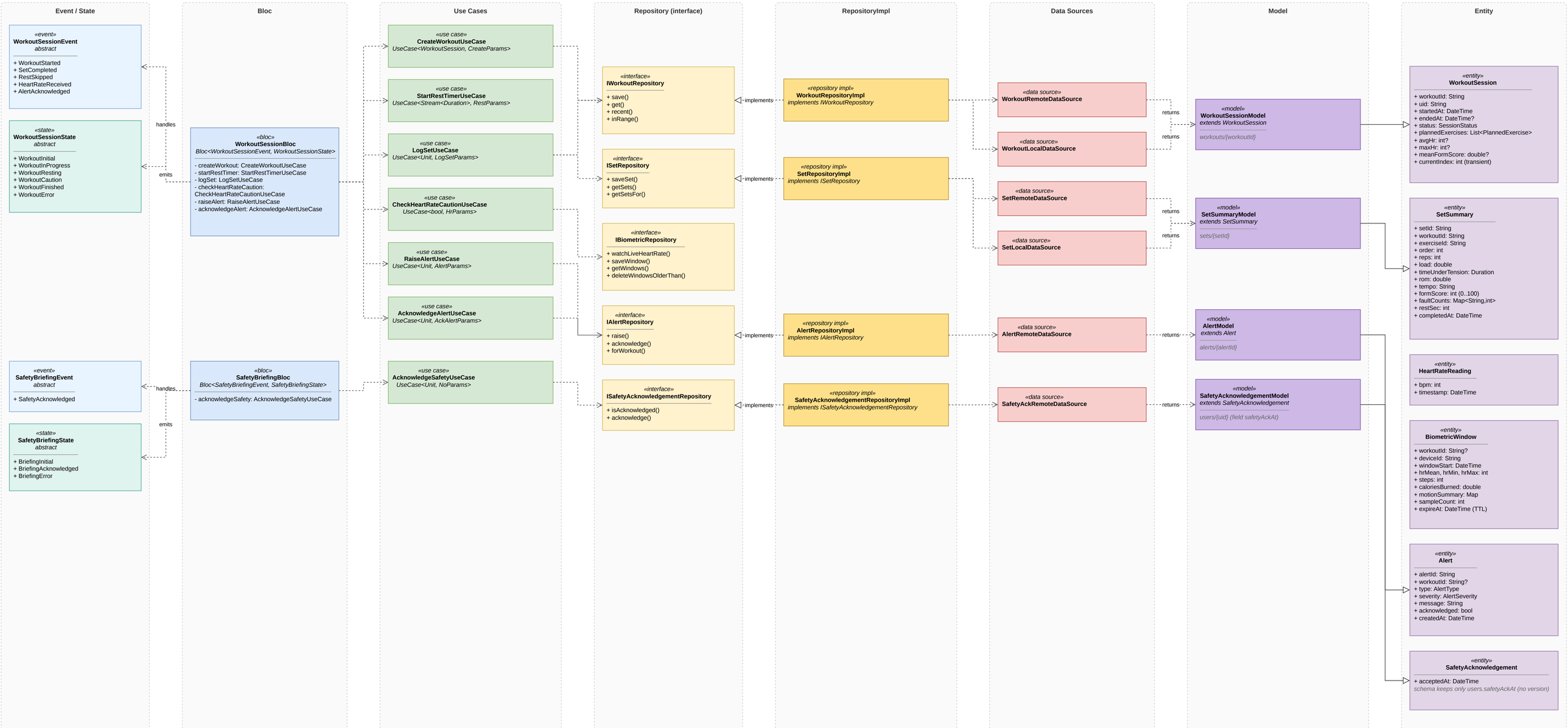
2 Pose Coaching

On-device pose analysis, cues, model selection



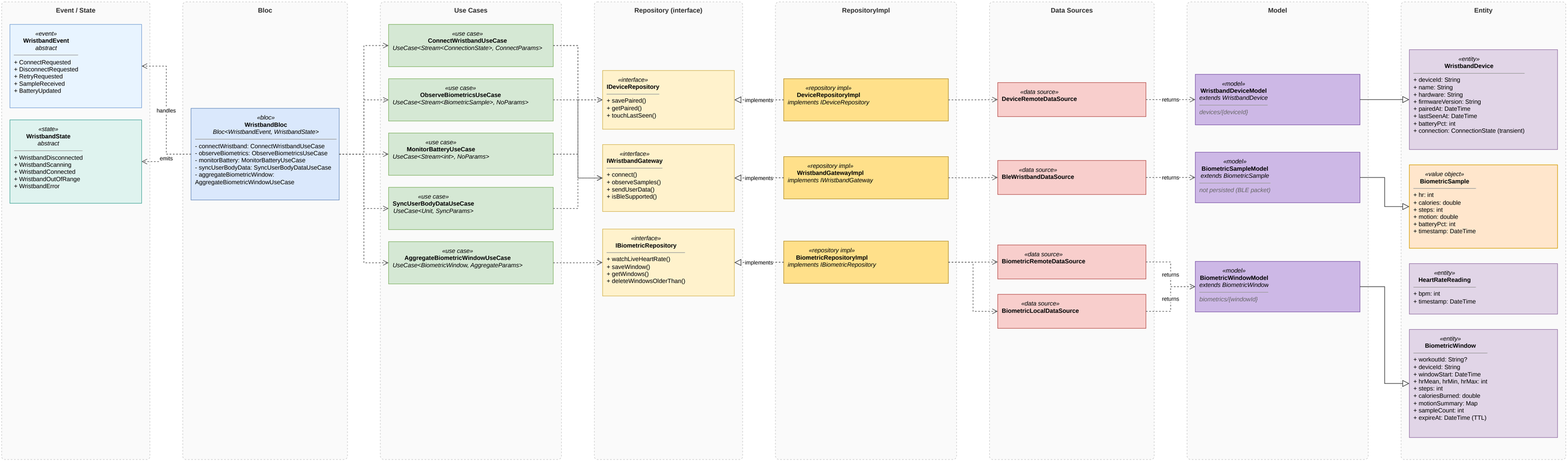
3 Workout Session and Safety

Workouts, sets, rest timer, heart-rate caution, alerts, safety acknowledgement



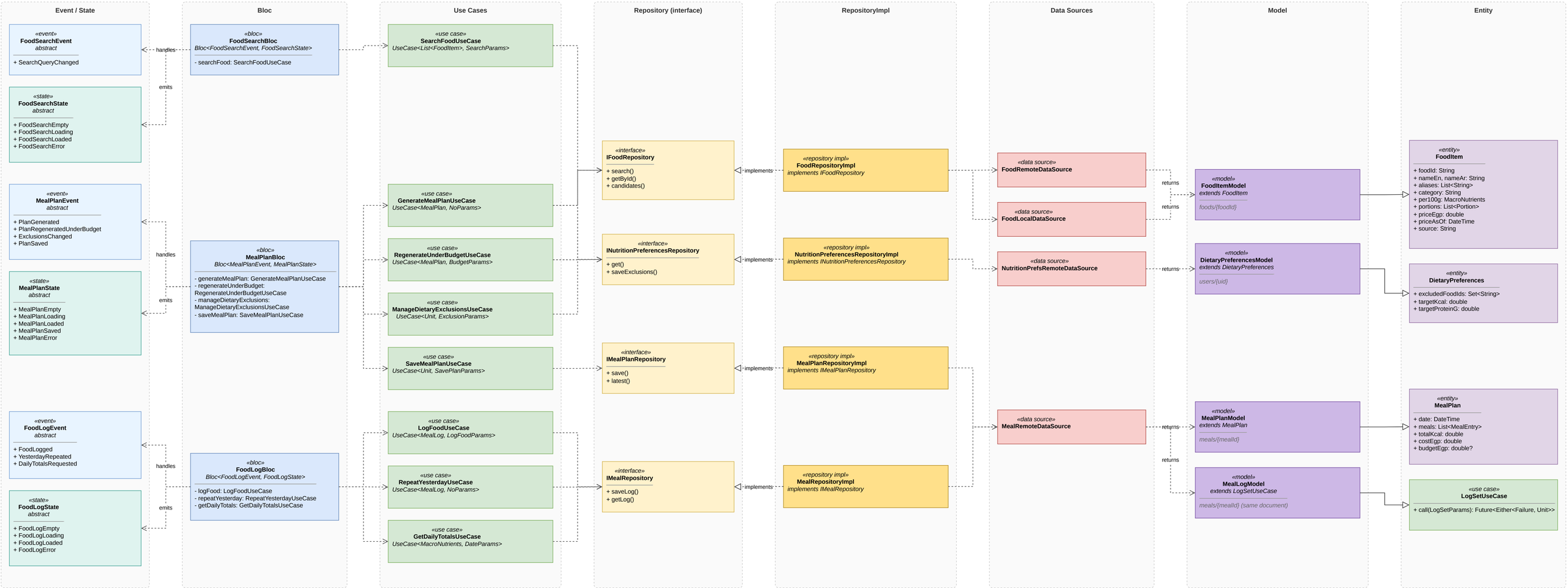
4 Wristband and Biometrics

BLE wristband, live samples, 60 s biometric windows, paired devices



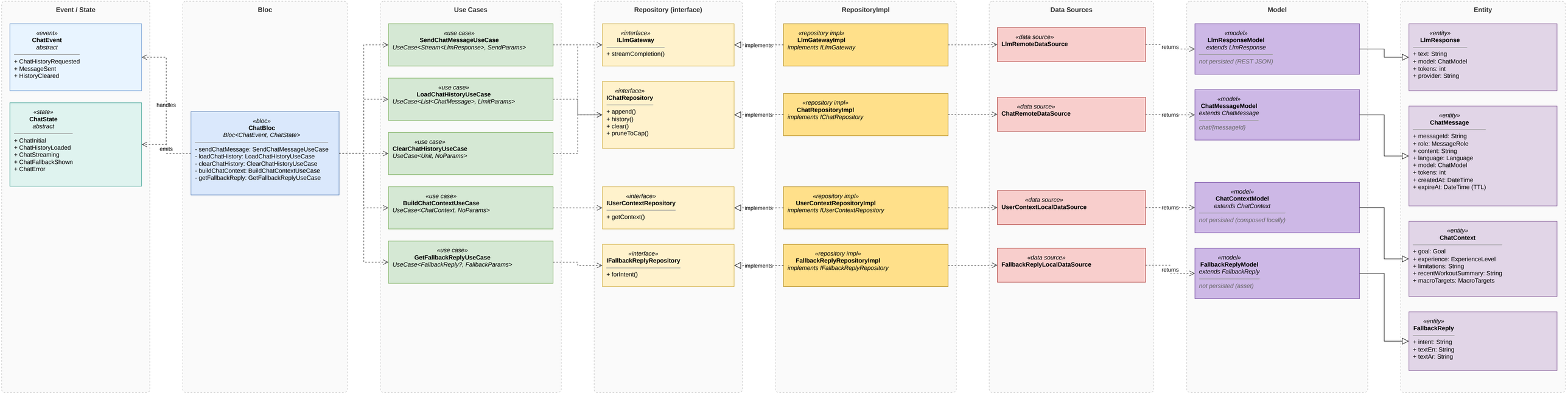
5 Nutrition and Meal Planning

Food catalogue, search, meal plans, daily log



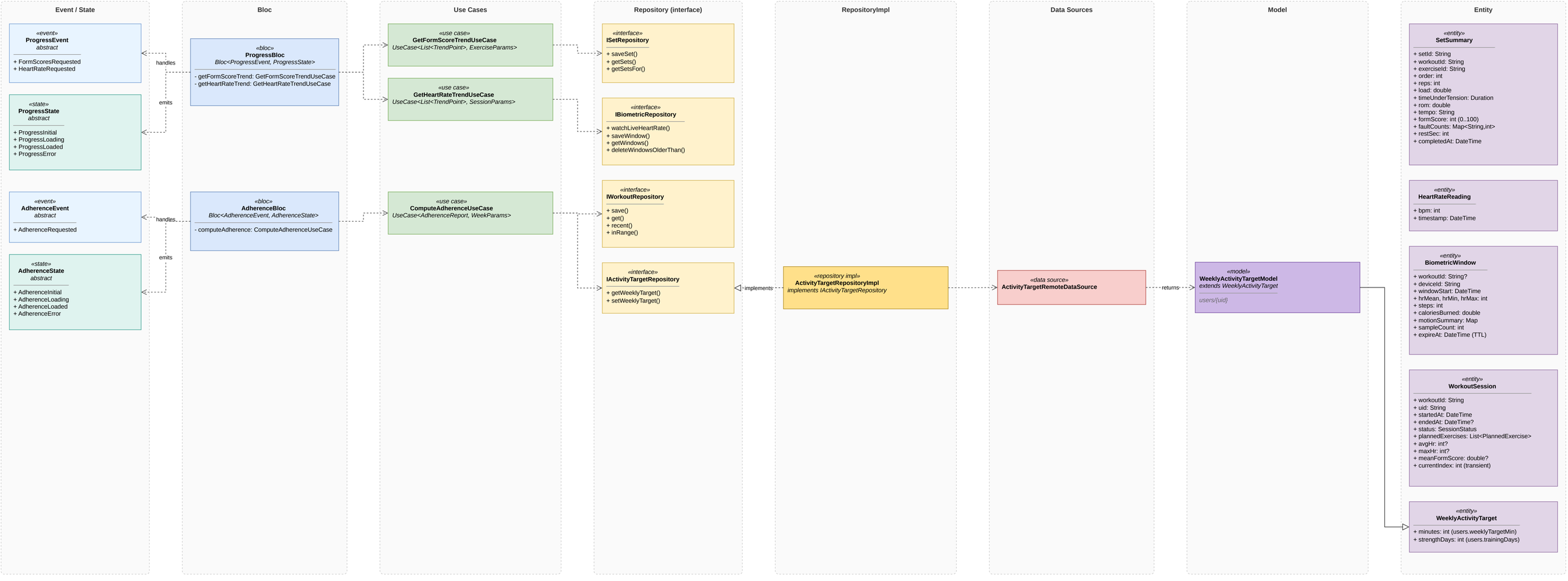
6 Chat Assistant

Domain-restricted bilingual assistant with templated fallback



7 Progress Analytics

Trends and adherence from stored sets, windows and workouts



8 Settings and Compliance

Language / numerals, consent, biometric retention

